HR Analytics Data Pipeline using Apache Spark

GitLab Repo: here

1. Student Details

Student Name	Email ID	Student ID	Responsibility
Henry Unachukwu	henry.unachukwu2@mail.dcu.ie	A00095735	Developed the Apache Spark pipeline, handling data ingestion, transformations, dataset joins, and analytical table creation. Managed Spark execution, performance handling and GitLab version control and contributed to technical sections of the report and results in the video walkthrough.
Ashika Rajkumar	ashika.rajkumar2@mail.dcu.ie	A00052851	Designed the pipeline structure and feature plan and carried out validation of Spark transformations and analytical outputs. Contributed for the report writing, presentation design and presented the problem motivation, dataset background and technology justification in the video walkthrough.

2. Dataset(s)

S/N	Dataset and Author	Dataset
1	HR Analytics Dataset by Shodolamu Opeyemi on Kaggle	https://www.kaggle.com/datasets/hopesb/hr-analytics-dataset/data
2	Payroll Dataset by Administration & Regulatory Affairs Department (ARA) on City of Houston Open Data	https://data.houstontx.gov/dataset/payroll

3. Video Demonstration

https://drive.google.com/file/d/1KMrRjPtamJtE4GbHbzJra9eDDz6Gl_1D/view?usp=sharing

4. Introduction and Motivation

This project introduces a cloud-based HR analytics data pipeline built with Apache Spark. It simulates a real-world enterprise where workforce and payroll data are kept in separate systems. The main goal is to combine different HR and Payroll data into a single, analytics-ready dataset that supports key HR metrics like workforce demographics, pay structure, tenure, attrition, and training results.

Many organisations keep HR and Payroll systems separate, using different formats and structures. These systems are usually set up for daily operations, not analysis. As a result, it is hard to get consistent insights across the organisation, such as understanding links between demographics, engagement, pay, and turnover. Our project aims to solve this by building a Spark-based data pipeline that connects and standardises these sources.

As part of the design phase, we reviewed several cloud data processing technologies covered in the module. Apache MapReduce, Hive, Spark and Storm. Storm was excluded because it is optimised for real-time streaming workloads such as sensor feeds or event logs, whereas our inputs are static CSV files. MapReduce, while reliable, incurs significant overhead due to its disk-based architecture, making it less suitable for iterative HR analytics and complex feature engineering. Hive offers SQL-like querying over large datasets but is less flexible for multi-step transformations such as custom feature engineering, conditional mappings and join logic across heterogeneous sources.

We chose Apache Spark as our main processing tool because it offers several advantages:

It uses in-memory distributed processing, which makes repeated data transformations and summaries much faster.

- A unified API combining DataFrame, SQL and functional transformations in Python (PySpark)
- It supports data cleaning and feature engineering, such as parsing dates, grouping data, and handling complex joins
- Spark can scale easily from a local setup to cloud clusters without needing to change the architecture
- It is also widely used in modern data engineering and HR analytics jobs

The Spark pipeline does more than just basic cleaning. It standardises data formats, converts date fields, calculates age and tenure, unifies race and gender categories, groups departments, and creates several summary tables. The final results are structured tables ready for reporting and visualisation, similar to what enterprise HR analytics platforms produce.

This project is driven by both academic and practical goals.To show skill in designing and building a scalable, cloud-based data processing pipeline with Spark. To create a realistic HR analytics scenario where large, varied datasets are combined to produce useful insights for organisations.

5. Related Work

Many organisations now use integrated HR analytics systems to bring together workforce data, automate reporting, and help with decision-making. These platforms offer strong analytics features, but most are closed systems with limited options for custom data engineering. Here is a short overview of three popular platforms.

a. Workday Human Capital Management (HCM)

Workday HCM is a cloud-based platform that brings HR, payroll, talent, and workforce management together in one system. It gives users real-time data access, strong security, and AI-powered analytics like predictive insights and skills analysis. However, because Workday is a proprietary system, users cannot see how data is processed or transformed. This reduces transparency and limits custom data engineering to only what is allowed by the system.

b. BambooHR Analytics Platform

BambooHR is made for small and mid-sized organisations and offers simple HR reporting on things like headcount, turnover, PTO, and EEO compliance. It has dashboards and custom reports that help reduce manual HR work. However, its analytics only cover data inside the platform, and users cannot do large custom transformations or add outside data beyond the built-in templates.

c. Sage People (AI-Powered Workforce Analytics)

Sage People delivers enterprise workforce analytics through the Salesforce platform, offering real-time dashboards on headcount, tenure, diversity, compensation, and organisational structure. It is well suited for large, global organisations. Because it relies on predefined Salesforce data flows, flexibility for custom data integration and pipeline redesign is limited, especially for large external dataset integration.

5.1 How Our Spark-Based Solution Differs

While these platforms have strong HR analytics features, our solution stands out in several key ways:

a. Full Transparency and Control of Pipeline

Commercial systems do not show their processing logic. In contrast, our solution builds the employee pipeline manually with Apache Spark, which includes:

- schema standardisation
- date parsing and normalisation
- age and tenure calculation
- demographic categorisation
- compensation breakdown
- payroll and HR data merging
- generation of analytical tables

This approach shows technical understanding and meets the assignment's requirement for meaningful preprocessing beyond just removing null values.

b. Open, Scalable Cloud Data Processing (Spark)

Workday, BambooHR, and Sage People work in closed systems. Our pipeline, however, uses open-source distributed computing with Apache Spark.

Spark allows:

- large dataset handling
- in-memory parallel processing
- scalable deployment (local or cloud clusters)
- reproducible academic experimentation

This directly supports your module's focus on cloud data engineering.

c. Cross-Platform Dataset Integration

Commercial systems only analyze data that is entered into their own platform.

Our project brings together two separate open datasets (HR and payroll) and combines them using domain-based categories, such as DepartmentCategory.

This reflects real-world data engineering challenges that enterprise tools often hide.

d. Customisable Analytics Output

Unlike proprietary dashboards, our Spark pipeline creates fully custom analytical tables, such as:

- Age, Gender and Race demographic cubes
- Tenure and engagement metrics
- Compensation summaries
- Department-level workforce distributions

This flexibility allows for deeper academic exploration than what BambooHR or Sage People offer.

6. Dataset Description

This project uses two large HR-related datasets that are publicly available from trusted open-data platforms. The datasets come from different sources, but each covers a different part of workforce and payroll information. Together, they are a good fit for showing how a cloud-based data processing pipeline can bring together different types of HR data.

6.1 Source of the Data

a. HR Analytics Dataset

The first dataset comes from Kaggle and has HR information for several thousand employees. It covers demographic details, job classifications, employment status, and engagement metrics.

Key Attributes Include:

- Employee demographic data (Gender, Race, Age, Marital Status)
- HR performance metrics such as engagement score, satisfaction score, and work-life balance
- Job data including department type, division, job title, and business unit
- Employment lifecycle details such as start date, exit date, tenure, and termination information
- Training data including program name, training duration, and training outcome

The dataset has 3,150 rows and more than 40 columns, offering a broad range of HR variables for workforce analytics and the dataset comes from the official Houston Government Open Data Portal, containing detailed payroll information for over 20,000 employees working for the City of Houston. This portal contains payroll information from 2018 to present.

b. Payroll Dataset
The payroll dataset provides information on City of Houston employee salaries and year-to-date pay.

Key Attributes Include:

- Employee compensation details such as base pay, overtime, gross pay, and total compensation
- Position information (Job title, Pay grade, Department name, Employment type)
- Employment dates such as position start and end dates
- Demographic indicators including gender, race, and birth year
- Status flags showing whether employees are active or inactive

This dataset offers the financial and employment details needed to complement the HR dataset.

6.2 Process of Extraction and Collection

- Both datasets were downloaded as CSV files and loaded into the pipeline with Apache Spark's DataFrame reader. This approach allowed for automatic schema detection and efficient processing
- HR and payroll datasets are usually kept in different systems, such as HRIS and payroll platforms. Keeping them separate in this project helped simulate a real-world enterprise data integration challenge
- The data was checked in advance to make sure it worked with Spark. This included using UTF-8 encoding, consistent delimiters, and proper numeric formats
- The schema was reviewed with df.printSchema() to see the column types before making any changes
- No manual changes were made to the raw files. All cleaning and transformations were done in Spark to keep the process reproducible

6.3 Why These Datasets Were Chosen
These datasets were selected because:
a. Large, real-world scale
After joining, the combined dataset has almost 3 million rows. This shows how well Spark can handle large-scale distributed processing.

b. Rich diversity of HR metrics
Data on engagement, performance, tenure, demographics, compensation, and departments make it possible to do meaningful analytics.

c. Heterogeneous structure
The differences between these datasets are similar to real challenges in enterprise data integration. This makes them a good fit for a cloud data engineering project.

d. Open and publicly accessible
These datasets meet the assignment guidelines, which require the use of public data.

7. Data Processing

This section explains how we process data with Apache Spark. It covers the steps for transforming data, aligning schemas, cleaning, and aggregating to build the final analytical dataset. You can find all related code in the project repository.

7.1 Overview of the Processing Workflow

Since the two datasets had different sources and structures, the main goal was to process them as follows:

- Load both datasets into Spark
- Clean and standardize each dataset, including data types, missing values, and date formats
- Create new HR analytics fields such as *Age*, *Age Group*, *Tenure*, and *Pay Grade Category*
- Join the datasets using a shared department mapping
- Handle duplicates from many-to-many relationships by using aggregation tables
- Produce final structured outputs that are ready for analysis and visualization outside Spark

The whole pipeline is built in Python with PySpark, showing how Spark can be used for large-scale data processing.

7.2 Start SparkSession

Below is the output from starting the SparkSession.

```
pip install pyspark

Requirement already satisfied: pyspark in c:\users\henry\anaconda3\lib\site
ages.

Requirement already satisfied: py4j==0.10.9.9 in c:\users\henry\anaconda3\
```

Step 1: Start SparkSession

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("HRAnalyticsPipline").getOrCreate()
spark
```

SparkSession - in-memory

SparkContext

Spark UI

Version	v4.0.1
Master	local[*]
AppName	HRAnalyticsPipline

Figure 1: Start SparkSession (Input and Output)

7.3 Loading the Datasets into Spark

We used Spark's distributed CSV reader to load both the HR and Payroll CSV files.

```
hr_df = spark.read.csv("HR_dataset.csv", header=True, inferSchema=True)
hr_df.show(5)
```

```
+----------+---------+--------+---------+--------+--------------------+-----
|Unnamed: 0|FirstName|LastName|StartDate|ExitDate|               Title|
+----------+---------+--------+---------+--------+--------------------+-----
|         0|    Uriah| Bridges|20-Sep-19|    NULL|Production Techni...|   Pe
|         1|    Paula|   Small|11-Feb-23|    NULL|Production Techni...|Renee
|         2|   Edward|    Buck|10-Dec-18|    NULL|  Area Sales Manager| Crys
|         3|  Michael| Riordan|21-Jun-21|    NULL|  Area Sales Manager| Rebe
|         4|  Jasmine|   Onque|29-Jun-19|    NULL|  Area Sales Manager|
+----------+---------+--------+---------+--------+--------------------+-----
only showing top 5 rows
```

Figure 2: Loading the Datasets for HR Dataset (Input and Output)

```
pay_df = spark.read.csv("payroll-2025.csv", header=True, inferSchema=True)
pay_df.show(5, truncate=False)
```

```
+----------------------------------------+----------------------
|Hash1                                   |Hash2
+----------------------------------------+----------------------
|C99E0C04311A1FBD86A68E01527B3D4F9CFAF656|8FBD6E88FE846F798156B5
|F224E67D62558473D8FD5F0332489905C592CDB5|4F2E3C274FFDC92F8436C9
|097646E986686B036D654E005F9581BE32377675|D9EFEE57EBA8157AAEBE1F
|2010C72D5CEE0AE8FA62F194DCEA2C1BAF1211A0|A4B01138B5CF79624949F9
|51A39920542A5C29D48B4EE8955CAEB3F7FF5322|50FAA8C948DD01067D2383
+----------------------------------------+----------------------
only showing top 5 rows
```

Figure 3: Loading the Datasets for Payroll Dataset (Input and Output)

7.4 Cleaning and Transformation

a. Handling Missing Values

Handling missing values was a key part of the process. For example, the *ExitDate* column in the HR dataset and the *Date_Position_Ended* column in the payroll dataset had nulls to show employees who were still active. We kept these nulls because they have business value. Later, we used these nulls to create a *TurnoverFlag* in HR and a *PositionEndFlag* in Payroll.

For non-critical fields like Training Outcome or email, we either left missing values as they were or grouped them as *Unknown*. This helped keep the data accurate for analysis.

```
+----------+---------+--------+---------+--------+-----+----------+-----
|Unnamed: 0|FirstName|LastName|StartDate|ExitDate|Title|Supervisor|ADEma
+----------+---------+--------+---------+--------+-----+----------+-----
|         0|        0|       0|        0|    1544|    0|         0|
+----------+---------+--------+---------+--------+-----+----------+-----
```

Figure 4: Handling Missing Values for HR Dataset (Output)

```
---+-------------------+------------------+---------+--
gan|Date Position Ended|Annual Base Salary|Gross Pay|Ba
---+-------------------+------------------+---------+--
  0|              22462|                 0|        0|
---+-------------------+------------------+---------+--
```

Figure 5: Handling Missing Values for Payroll Dataset (Output)

b. Standardizing Categorical Fields

We took steps to make sure the datasets were consistent.

- We standardized department names by trimming spaces, using lowercase letters, and removing extra whitespace
- We made sure the race and gender categories matched in both datasets
- We added new columns, such as *DeptCategory* and *GenderCode*, to keep the data consistent

```
After cleaning column names:
root
 |-- Unnamed:_0: integer (nullable = true)
 |-- FirstName: string (nullable = true)
 |-- LastName: string (nullable = true)
 |-- StartDate: string (nullable = true)
 |-- ExitDate: string (nullable = true)
 |-- Title: string (nullable = true)
 |-- Supervisor: string (nullable = true)
 |-- ADEmail: string (nullable = true)
```

Figure 6: Standardizing Categorical Fields for HR Cleaned Dataset (Output)

```
After cleaning column names:
root
 |-- Hash1: string (nullable = true)
 |-- Hash2: string (nullable = true)
 |-- Birth_Year: integer (nullable = true)
 |-- Gender: string (nullable = true)
 |-- Race: string (nullable = true)
 |-- Pay_Grade: string (nullable = true)
 |-- FLSA_Status: string (nullable = true)
```

Figure 7: Standardizing Categorical Fields Payroll Cleaned Dataset (Output)

c. Date Formatting & Derived Columns

The HR dataset did not include an age column, so we used other available data to calculate age.

```
After converting date columns:
+----------+--------+----------+-------------+-----------+
| StartDate|ExitDate|       DOB|Training_Date|Survey_Date|
+----------+--------+----------+-------------+-----------+
|2019-09-20|    NULL|1969-10-07|   2023-07-15| 2023-01-14|
|2023-02-11|    NULL|1965-08-30|   2022-09-12| 2022-09-09|
|2018-12-10|    NULL|1991-10-06|   2022-08-13| 2023-05-27|
|2021-06-21|    NULL|1998-04-04|   2022-12-15| 2023-06-16|
|2019-06-29|    NULL|1969-08-29|   2023-07-13| 2022-11-25|
+----------+--------+----------+-------------+-----------+
only showing top 5 rows
```

Figure 8: Date Formatting for HR Clean Dataset (Output)

```
After converting date columns:
+-------------------+-------------------+
|Date_Position_Began|Date_Position_Ended|
+-------------------+-------------------+
|         2025-01-01|         2025-02-01|
|         2025-01-01|         2025-02-01|
|         2025-01-01|         2025-02-01|
|         2025-01-01|         2025-02-01|
|         2025-01-01|         2025-02-01|
+-------------------+-------------------+
only showing top 5 rows
```

Figure 9: Date Formatting for Payroll Clean Dataset (Output)

```
Age column sample:
+----------+---+
|       DOB|Age|
+----------+---+
|1969-10-07| 56|
|1965-08-30| 60|
|1991-10-06| 34|
|1998-04-04| 27|
|1969-08-29| 56|
+----------+---+
only showing top 5 rows
```

Figure 10: Derived Columns for Age - HR Dataset (Output)

```
+---------+-----+
|AgeGroup|count|
+---------+-----+
|   30-39|  550|
|     60+| 1315|
|   40-49|  512|
|Under 30|  316|
|   50-59|  457|
+---------+-----+
```

Figure 11: Derived Columns for Age groups - HR Dataset (Output)

```
Tenure column sample:
+----------+------+
| StartDate|Tenure|
+----------+------+
|2019-09-20|     6|
|2023-02-11|     2|
|2018-12-10|     6|
|2021-06-21|     4|
|2019-06-29|     6|
+----------+------+
only showing top 5 rows
```

Figure 12: Derived Columns for Employee's Tenure based on their Start and Exit Dates - HR Dataset (Output)

```
Turnover flag sample:
+---------+------------+
|ExitDate|TurnoverFlag|
+---------+------------+
|    NULL|           0|
|    NULL|           0|
|    NULL|           0|
|    NULL|           0|
|    NULL|           0|
+---------+------------+
only showing top 5 rows
```

Figure 12: Derived Columns for Turnover Flag (Output)

```
Final cleaned data for HR data:
+---------+--------+----------+--------+----------------------+----
|FirstName|LastName|StartDate |ExitDate|Title                 |Busi
+---------+--------+----------+--------+----------------------+----
|Uriah    |Bridges |2019-09-20|NULL    |Production Technician I|CCDR
|Paula    |Small   |2023-02-11|NULL    |Production Technician I|EW
|Edward   |Buck    |2018-12-10|NULL    |Area Sales Manager     |PL
|Michael  |Riordan |2021-06-21|NULL    |Area Sales Manager     |CCDR
|Jasmine  |Onque   |2019-06-29|NULL    |Area Sales Manager     |TNS
+---------+--------+----------+--------+----------------------+----
only showing top 5 rows
```

Figure 13: Dropped Useless Columns (Output)

d. Creating Payroll Processing Fields

The payroll dataset contains detailed information about employee compensation.
We calculated the total compensation for each employee.

```
Total compensation sample:
+--------+------------+---------+------------------+
|Base_Pay|Overtime_Pay|Other_Pay| TotalCompensation|
+--------+------------+---------+------------------+
| 1192.32|         0.0|    94.29|           1286.61|
| 1812.91|         0.0|     77.7|1890.6100000000001|
| 3166.68|         0.0| 94842.41|          98009.09|
|  1090.2|       290.3|   797.88|           2178.38|
| 1571.92|         0.0| 19156.29|          20728.21|
+--------+------------+---------+------------------+
only showing top 5 rows
```

Figure 14: Create Column for Employee Total Compensation (Output)

```
+---------+----------------+
|Pay_Grade|PayGradeCategory|
+---------+----------------+
|     15.0|             Mid|
|     20.0|             Mid|
|     27.0|            High|
|      6.0|             Low|
|     13.0|             Mid|
+---------+----------------+
only showing top 5 rows
```

Figure 15: Create Column for Pay Grade Categories, i.e. Low, Mid and High (Output)

```
+----------+----+
|Birth_Year| Age|
+----------+----+
|    1955.0|70.0|
|    1955.0|70.0|
|    1963.0|62.0|
|    1973.0|52.0|
|    1959.0|66.0|
|    1958.0|67.0|
|    1959.0|66.0|
|    1948.0|77.0|
|    1960.0|65.0|
|    1954.0|71.0|
+----------+----+
only showing top 10 rows
```

Figure 16: Create Column for Employee's Age (Output)

7.5 Merging the Datasets

We merged the datasets by creating a mapping at the department level:

- Departments in HR dataset: *DeptCategory*
- Departments in Payroll dataset: *Department_Name* (cleaned and mapped) to *DeptCategory*

```
HR Categories:
+------------+
|DeptCategory|
+------------+
|       Other|
|  Technology|
|   Corporate|
|  Operations|
+------------+
```

```
Payroll Categories:
+--------------+
|  DeptCategory|
+--------------+
|Public Service|
|    Technology|
|     Corporate|
|    Operations|
+--------------+
```

Figure 17: Print the Department Categories for HR and Payroll Cleaned Dataset (Output)

Step 10: Perform Merge

```python
final_df = hr_with_cat.join(
    pay_with_cat,
    on="DeptCategory",
    how="inner"
)
```

```python
final_df.show(5, truncate=False)
```

Figure 18: Merging the Datasets (Input)

The join grew much larger because each department had many employees in both datasets, which led to many-to-many relationships.

After joining, the dataset grew to almost 3 million rows. This was expected since both datasets shared departments. Since our goal was analytics rather than matching individual employees, we handled duplicates by creating aggregated analytical tables.

7.6 Creation of Analytical Summary Tables

These summary tables show the final, processed results from the Spark pipeline.

AgeGroup	Gender	Race	EmployeeCount
40-49	Female	White	27920
40-49	Female	Black	137120
40-49	Female	Asian	25550
40-49	Male	Hispanic	107865
50-59	Female	Hispanic	55545
60+	Female	Asian	19260

Figure 19: Workforce Demographics (Output)

DeptCategory	AgeGroup	Gender	Race	Avg_Engagement	Avg_Satisfac
Technology	40-49	Female	Hispanic	3.010344827586207	3.0620689655
Corporate	Under 20	Female	Other	3.0272727272727273	2.6545454545
Technology	60+	Male	Hispanic	3.010344827586207	3.0620689655
Operations	60+	Female	White	3.0	3.1478260869
Corporate	Under 20	Female	Asian	3.0272727272727273	2.6545454545
Corporate	60+	Male	White	3.0272727272727273	2.6545454545

Figure 20: Department-Level Engagement Metrics (Output)

DeptCategory	AgeGroup	Gender	Race	Avg_Salary	Avg_BasePay
Technology	40-49	Female	Hispanic	74320.25	44631.5275000002
Corporate	Under 20	Female	Other	29120.0	2518.8333333333335
Technology	60+	Male	Hispanic	94340.09090909091	32295.591818180535
Operations	60+	Female	White	90946.48979591837	41959.45081632884
Corporate	Under 20	Female	Asian	29120.0	2250.7916666666665

Figure 21: Compensation Summary (Output)

DeptCategory	AgeGroup	Gender	Race	Total_Leavers	Total_Employees
Technology	40-49	Female	Hispanic	1240	2320
Corporate	Under 20	Female	Other	213	330
Technology	60+	Male	Hispanic	3410	6380
Operations	60+	Female	White	8379	16905
Corporate	Under 20	Female	Asian	852	1320

Figure 22: Turnover Rate Summary (Output)

```
+------------+--------+------+--------+------------------+---------------
|DeptCategory|AgeGroup|Gender|Race    |Avg_TrainingDays  |Total_TrainingC
+------------+--------+------+--------+------------------+---------------
|Technology  |40-49   |Female|Hispanic|3.0120689655172415|1329740.92
|Corporate   |Under 20|Female|Other   |2.963636363636364 |181869.48000000
|Technology  |60+     |Male  |Hispanic|3.0120689655172415|3656787.5300000
|Operations  |60+     |Female|White   |2.907246376811594 |9072758.6600000
```

Figure 23: Training Outcomes Summary (Output)

7.7 Exporting Final Analytical Results

```
table1_demo.coalesce(1).write.option("header", True).mode("overwrite").csv("analytics_output/demo")
table2_engagement.coalesce(1).write.option("header", True).mode("overwrite").csv("analytics_output/engagement")
table3_compensation.coalesce(1).write.option("header", True).mode("overwrite").csv("analytics_output/compensation")
table4_turnover.coalesce(1).write.option("header", True).mode("overwrite").csv("analytics_output/turnover")
table5_training.coalesce(1).write.option("header", True).mode("overwrite").csv("analytics_output/training")
```

Figure 24: Spark Export of each Table as a CSV file (Input)

These CSV exports can be used in external visualization tools like Excel, Tableau or Power BI.

8. Development of the Pipeline

This section explains how we developed our cloud-based pipeline with Apache Spark, covering data extraction, processing, and creating analytical outputs.

8.1 Environment Setup For Local Development of PySpark on Windows

We set up an Apache Spark environment on Windows 11 to develop the HR analytics pipeline.

PySpark, the Python library for Spark, let us use Spark within the Jupyter Notebook. While cloud platforms like AWS EMR or Databricks are common for distributed data processing, we chose a local setup for easier development, reproducibility, and faster testing before deployment.

Below are the steps taken:

a. Install Python
We downloaded and installed Python from the official website here.

b. Installation of Java and Environment Variables
We downloaded and installed Java Development Kit (JDK) 21 x64 from Oracle's website here.

Configured environment variables:

- Name: JAVA_HOME
- Value: C:\Program Files\Java\jdk-21

This setup lets Spark access the Java virtual machine (JVM) needed by the Python 3 Kernel for communication between Python and Spark.

c. WinUtils Setup
Spark needs Hadoop's native binaries on Windows. We set up the following:

- We downloaded the compatible version of winutils.exe for our Hadoop version here
- Added to C:\hadoop\bin\winutils.exe
- SetWe added it to C:\hadoop\bin\winutils.exe.=C:\hadoop

d. Restarted the Computer

e. PySpark Configuration

```
pip install pyspark
```

Figure 25: Installed PySpark

```
from pyspark.sql import SparkSession
spark = SparkSession.builder.appName("HRAnalyticsPipline").getOrCreate()
spark
```

Figure 26: Setup Spark on Jupyter Notebook

All Spark jobs, including transformations, merges, and exports, were started from within Jupyter.

8.2 Data Extraction Tool

The data extraction step used Apache Spark's Distributed DataFrame Reader to load CSV files into distributed memory.

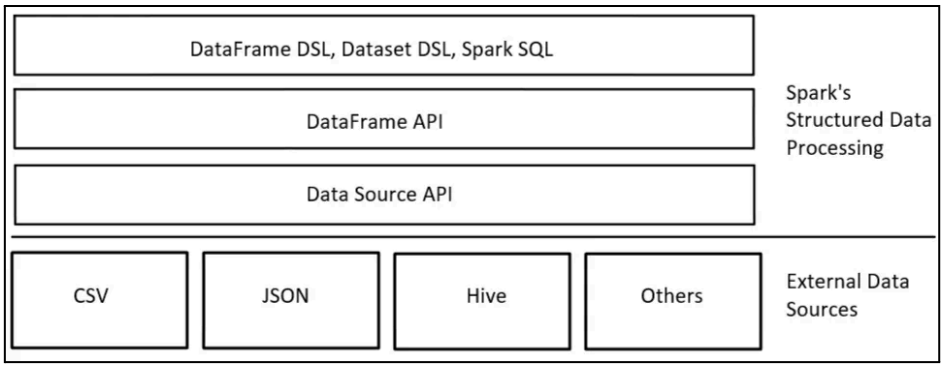

Figure 27: Spark Architecture, Connectors and Data Sources

Spark as the Extraction Tool

Spark's extraction engine was chosen for several reasons:

- It can read CSV files with millions of rows directly into distributed memory, without needing to load the whole file into RAM
- Spark can automatically detect data types using schema inference
- It reliably manages variable-width text fields, missing values, and different encodings
- Spark supports cloud storage formats like S3, HDFS, and Azure Blob, making it easy to extend to a cloud environment in the future

8.3 Data Processing Tool

The main part of the pipeline used Apache Spark (PySpark) to handle all the transformation steps needed to combine the HR and Payroll datasets. Processing included cleaning, standardizing, feature engineering, merging, and creating summary tables for later reporting.

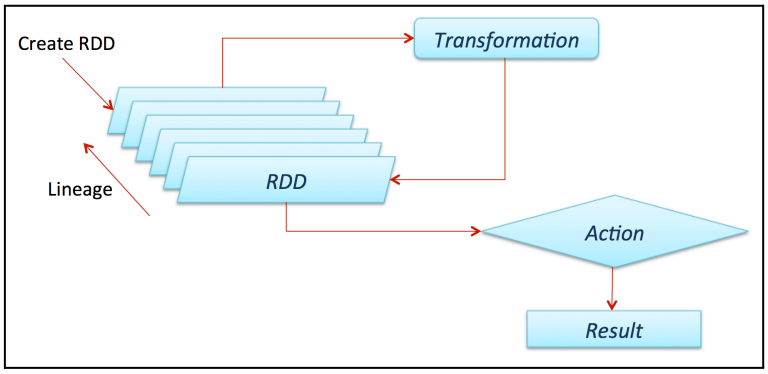

Figure 28: Transformation process in Apache Spark

Why Spark for Processing
Spark was picked over MapReduce, Hive, or Storm for several reasons:

- It does in-memory distributed computing, which is much faster for repeated HR analytics than MapReduce, which relies heavily on disk operations
- Spark offers a single API for SQL-style queries, functional transformations, and DataFrame operations all in one place
- It works well for batch analytics, while Storm is built for real-time streaming, which isn't needed for static HR or Payroll CSV files
- Spark efficiently manages large joins, aggregations, and feature engineering tasks
- Using Spark reflects how HR analytics systems work in industry, so this project matches real enterprise data engineering practices

8.4 Interface/Dashboard
We can use Tableau or Power BI for visualisation for several reasons:

- We are not allowed to use Pandas, Matplotlib or seaborn in the Spark notebook
- Tableau could be used to create professional charts for HR analytics, such as bar charts, heatmaps and demographic pyramids
- This approach keeps our results reproducible and in line with industry reporting standards
- We can use external tools for reporting, while Spark was only required for data processing

We could create the following visualisations:

- Age, Gender and Race heatmaps
- Department compensation comparisons
- Turnover rate bar charts
- Training cost distributions

The CSV files exported from Spark can be used for these visualizations.

9. Challenges and Lessons Learned
Building a cloud-based HR analytics pipeline with Apache Spark brought several technical and organisational challenges that shaped our project. We faced these issues during dataset selection, environment setup, data engineering, integration, and result validation. Solving them gave us a better understanding of distributed processing, schema harmonisation, and merging different public datasets.

a. Environment & Spark Setup Challenges
Our first challenge was setting up Spark on Windows. We ran into problems like Java version mismatches, path errors, and Spark not launching because the Java gateway process exited unexpectedly.

We also saw Spark warnings about missing `winutils.exe`, which meant we had to find manual solutions.

What we learned from this was:

- Spark does not work easily on Windows, so setting up the JVM correctly is important. We gained hands-on experience with distributed tools and learned how to fix cluster launch errors

b. Working Without Pandas or Traditional Python Libraries
A major constraint was the requirement to avoid Pandas, Matplotlib or other common Python libraries and rely solely on Spark for all transformations. This made tasks such as date parsing, missing value handling and feature engineering more complex because Spark DataFrames behave differently from Python DataFrames.

From this, we learned that:

- We got better at using native Spark functions like withColumn, to_date, when, and groupBy. We also learned to approach problems with a distributed, SQL-style mindset instead of row-by-row Python logic

c. Dataset Incompatibility and Schema Misalignment
The HR dataset and the Houston Payroll dataset were from different organisations. They had no shared identifiers, used different column names and date formats, and had different department structures. We could not merge them directly.

What we learned from this was:
- We saw how important it is to harmonise schemas, group domains, and create a common integration layer. By building a shared "DeptCategory" mapping, we learned how big analytical systems merge organisational datasets without using individual-level identifiers

d. Performance Constraints During Complex Operations
Some operations particularly joins, date parsing and aggregations caused Python worker crashes due to memory pressure in Jupyter. Spark's lazy evaluation sometimes made debugging more difficult until actions like count() were executed.

From this, we learned that:
- We learned ho w to optimise Spark jobs, avoid unnecessary computation, and break long pipelines into checkpoints. We also figured out how to handle distributed job failures and read Spark exception logs

e. Visualisation Challenges
Spark does not support native visualisation and we were restricted from using Python plotting libraries inside the pipeline. This required exporting aggregated tables and performing visualisation externally.

What we learned from this was:
- Spark is great for data processing, but it is not designed for making presentations. In real industry systems, analytics pipelines usually send their outputs to BI tools like Excel or Tableau, which matches real-world practice

10. Reference

[1]. Administration & Regulatory Affairs Department (ARA), City of Houston Payroll Dataset, Houston Open Data Portal (2025)
https://data.houstontx.gov/dataset/payroll

[2]. AmpCode (2023) Install Apache PySpark on Windows PC
https://www.youtube.com/watch?v=OmcSTQVkrvo&list=PL6UwySlcwEYKiC-EjEmN4f33c5fVpbzha&index=2

[3]. BambooHR, HR Reporting & Analytics
https://www.bamboohr.com/hr-software/hr-reporting

[4]. Hadoop WinUtils for Windows
https://github.com/kontext-tech/winutils/blob/master/hadoop-3.4.0-win10-x64/bin/winutils.exe

[5]. MachineLearningPlus, *How to Install PySpark on Windows.*
https://www.machinelearningplus.com/pyspark/install-pyspark-on-windows/

[6]. Patel, Chitrarth (2022). "Spark DataFrame API - Fundamentals". Medium.
https://medium.com/%40chitrarth236/spark-dataframe-api-part-1-fundamentals-41dd51e46714
[7]. PySpark Installation Guide (Official) & Apache Spark SQL, DataFrameReader Documentation.

https://spark.apache.org/docs

[8]. Sage People, HR and People Analytics
https://www.sage.com/en-ie/sage-business-cloud/people/product-capabilities/hr-people-analytics/

[9]. Shodolamu Opeyemi, HR Analytics Dataset, Kaggle (2023)
https://www.kaggle.com/datasets/hopesb/hr-analytics-dataset/data

[10].StackOverflow (2016). Transformation process in Apache Spark. Available at:
https://stackoverflow.com/q/39293088

[11]. Workday, Three Reasons Why Workday Human Capital Management Enables Modern HR (2025)
https://forms.workday.com/en-gb/ebooks/why-workday-hcm-enables-modern-hr/form.open.html/dl/1